

# Design and Optimization of Combinational Digital Circuits Using Modified Evolutionary Algorithm

Adam Słowik and Michał Białko

Department of Electronic, Technical University of Koszalin,  
ul. Śniadeckich 2, 75-453 Koszalin, Poland  
adamslowik@go2.pl

**Abstract.** In this paper possibility of design and optimization of combinational digital circuits using modified evolutionary algorithm is presented. Modification of evolutionary algorithm depends on introduction of multilayer chromosomes and genetic operators operating on them. Design results for four combinational circuits obtained using this method are compared with described in literature methods: Karnaugh Maps, Quine-McCluskey and NGA and MGA genetic algorithms. Described evolutionary algorithm leads in many cases to better results.

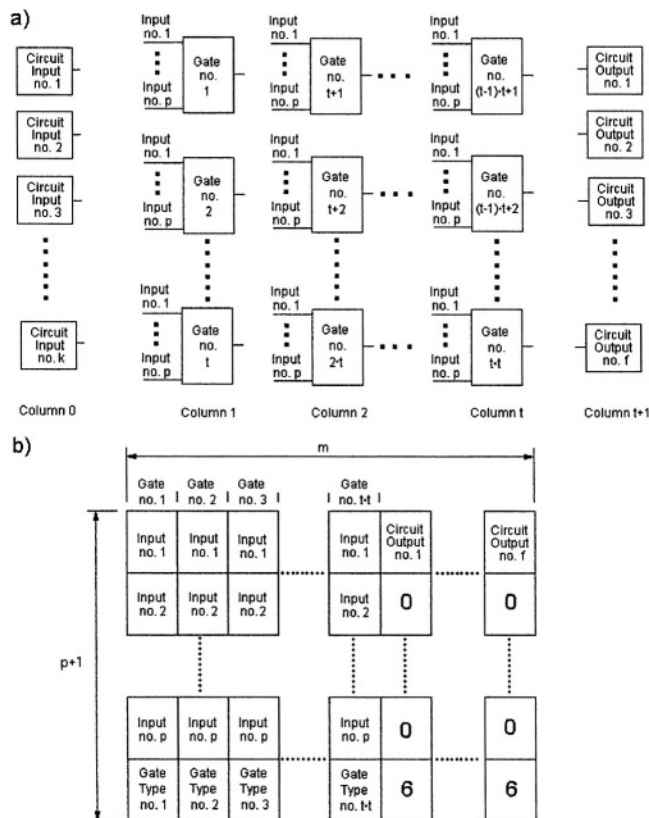
## 1 Introduction

Among design methods of combinational circuits, two of them are most popular: Karnaugh Maps [1] and Quine-McCluskey method [2] [3]. Design of combinational digital circuits based on these methods uses usually only gates NOT, AND, OR. Also many different methods useful for evolutionary design of digital circuits were created. Among them we can enumerate Miller [4], Kalganova [5] and Coello [6][9] works where NGA algorithm (*Genetic Algorithm with N-cardinality representation*) was described. In his work [7] Coello has proposed multiobjective version of algorithm useful for digital circuits design. The main idea of MGA algorithm (*Multiobjective Genetic Algorithm*) is a utilisation of multiobjective optimization (similarly as in VEGA [8] system) to solve optimization problem consisting of  $k$  equality constraints. In this paper evolutionary algorithm useful for design and optimization (with regard to gate number) of combinational digital circuits is presented. In presented algorithm a new representation of multilayer chromosomes is introduced. Compared to existing algorithms, multilayer chromosomes with  $p+1$  layers ( $p$  - number of gate inputs) and  $m$  genes in each layer are introduced. This algorithm is named MLCEA (*Multi-Layer Chromosome Evolutionary Algorithm*). For multilayer structure of chromosome specially designed crossover and mutation operators with suitable repair procedures are introduced. Results obtained using the proposed MLCEA algorithm are compared with those obtained by: Karnaugh Maps (KM), Quine-McCluskey method (QM) and NGA and MGA algorithms.

## 2 Evolutionary MLCEA Algorithm

### 2.1 Representation of Individuals (Potential Solutions)

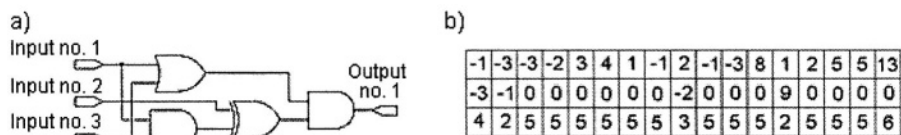
In order to create an initial population, we create a pattern (template) of designed circuit, shown in Fig. 1a, which structure is similar to the pattern used in work [7].



**Fig. 1.** The structure of circuit coding: a-pattern, b-chromosome

The structure and coding of chromosomes is shown in Fig. 1b. In the place marked as “Input no. x” in chromosome we put the number of the circuit input or the gate number from the pattern, which output is to be connected to this input; in the place “Gate Type no. x” we put one of the five digits, which represent respectively: 1-NOT gate, 2-AND gate, 3-XOR gate, 4-OR gate, 5-direct connection (DC) of this gate input with its output. Digit “0” represents lack of connection of this gate input in the pattern, and digit “6” represents the

circuit output. In the place “Output no. x” we put the pattern gate number, which output is to be connected to this circuit output. All circuit inputs are represent by negative numbers, that is the first input is represent by the number “-1”, etc. In Fig. 2 an example circuit (a) and corresponding to it chromosome (b) are presented; in this case pattern consists of  $4 \cdot 4 = 16$  gates.



**Fig. 2.** Example of digital circuit-a, corresponding to it multilayer chromosome-b

## 2.2 General Concept of Evolutionary Algorithm Operation

A goal of the algorithm is to find such a set of gates and connections between them in created pattern, so that the circuit fulfils all the constraints described by the truth table and is composed of the lowest number of gates. In the algorithm two objective function FC1 and FC2 are introduced. At the beginning we create an initial circuit population (randomly) avoiding feedback occurrence in the circuits, which consists of potential solutions (individuals - multilayer chromosomes). Then we evaluate a fitness according to the first objective function for each individual; in this step the algorithm finds circuits which fulfil a specified truth table (algorithm minimizes the function FC1). The function FC1 specifies the number of constraints that follow from the truth table, which are not fulfilled by a particular individual. Besides, we are looking for circuits that fulfil the constraints posed by the truth table, but with the highest number of gates allowed by the circuit pattern size. This treatment increases the search space in the second phase of the algorithm. In the case, when the value of  $FC1 \neq 0$  then modified roulette selection is used, which increases a number of the best solutions in a new created population. Then, crossover and simple mutation operators described in section 2.3 are used. In the case when the value of  $FC1 = 0$  then we compute the value of the objective function FC2, which determines the number of gates in the circuit (algorithm minimizes FC2 function value). Then, we check whether the algorithm converged (lack of changes of the best solution).

## 2.3 Genetic Operators and Repair Mechanism

In this work novel crossover and mutation operators operating on  $p+1$  layer chromosomes are introduced. One point crossover depends on cutting all  $p+1$  layers in one randomly chosen point. Thanks to this created new chromosome includes the gates with unchanged input structure. Simple mutation depends

on choosing of one gene of the chromosome to mutation with equal probability. In the case when to mutation a gene from the last  $p+1$  layer of chromosome is chosen, where the gate type number is written down, then an integer value from the range  $[1, 5]$  is randomly chosen, what corresponds to gate type changing. The genes of the  $p+1$  layer of the chromosome, but corresponding to the columns associated with circuit outputs are not used for mutation (they always have the value “6”). In the case when to mutation the gene from other layers is chosen, then one of connections between the gates in the pattern is changed. After mutation it is necessary to repair mutated chromosome. The repair mechanism depends on looking through all genes which are placed in the last  $p+1$  layer of chromosome, where the gate type is written down. If the gate type is AND, XOR, or OR, then we check if any input (from 1 to  $p$ ) of the gate is not connected (chosen gene belonging to this gate has value 0). In the case of its disconnection a new connection is randomly chosen. If the gate type is NOT, DC or represents particular circuit output, then we check if the first input of the gate is connected. In the case of its disconnection also a new connection is chosen and other inputs of these gates (from 2 to  $p$ ) are disconnected by writing down a digit “0”.

### 3 Description of Experiments

During performed experiments the parameters of evolutionary algorithm were: population size=100, crossover probability=0.5, mutation probability=0.05. For digital circuit realization two-input gates were chosen (comparative results based on such gates were available). In all experiments pattern size was chosen experimentally. In Table 1 the truth tables for each designed circuit are presented. Symbol “In” represent inputs, and symbol “O” corresponds to circuit outputs.

**Table 1.** Truth tables of designed circuits

| Circuit no. 1 |   |   |   | Circuit no. 2 |   |   |   |   | Circuit no. 3 |    |   |   |   | Circuit no. 4  |                |                |                |                |                |                |   |  |
|---------------|---|---|---|---------------|---|---|---|---|---------------|----|---|---|---|----------------|----------------|----------------|----------------|----------------|----------------|----------------|---|--|
| In            |   |   | O | In            |   |   |   |   | O             | In |   |   |   | O              | In             |                |                |                |                | O              |   |  |
| X             | Y | Z | F | Z             | W | X | Y | F | A             | B  | C | D | F | A <sub>1</sub> | A <sub>0</sub> | B <sub>1</sub> | B <sub>0</sub> | X <sub>2</sub> | X <sub>1</sub> | X <sub>0</sub> |   |  |
| 0             | 0 | 0 | 0 | 0             | 0 | 0 | 0 | 1 | 0             | 0  | 0 | 0 | 1 | 0              | 0              | 0              | 0              | 0              | 0              | 0              |   |  |
| 0             | 0 | 1 | 0 | 0             | 0 | 0 | 1 | 1 | 0             | 0  | 0 | 1 | 0 | 0              | 0              | 0              | 1              | 0              | 0              | 1              |   |  |
| 0             | 1 | 0 | 0 | 0             | 0 | 1 | 0 | 0 | 0             | 0  | 1 | 0 | 0 | 0              | 0              | 1              | 0              | 0              | 1              | 0              |   |  |
| 0             | 1 | 1 | 1 | 0             | 0 | 1 | 1 | 1 | 0             | 0  | 1 | 1 | 0 | 0              | 0              | 1              | 1              | 0              | 1              | 1              |   |  |
| 1             | 0 | 0 | 0 | 0             | 1 | 0 | 0 | 0 | 0             | 1  | 0 | 0 | 1 | 0              | 1              | 0              | 0              | 0              | 0              | 1              |   |  |
| 1             | 0 | 1 | 1 | 0             | 1 | 0 | 1 | 0 | 0             | 1  | 0 | 1 | 1 | 0              | 1              | 0              | 1              | 0              | 1              | 0              |   |  |
| 1             | 1 | 0 | 1 | 0             | 1 | 1 | 0 | 1 | 0             | 1  | 1 | 0 | 1 | 0              | 1              | 1              | 0              | 0              | 1              | 1              |   |  |
| 1             | 1 | 1 | 0 | 0             | 0 | 1 | 1 | 1 | 0             | 1  | 1 | 1 | 1 | 0              | 1              | 1              | 1              | 1              | 0              | 0              |   |  |
|               |   |   |   | 1             | 0 | 0 | 0 | 1 | 1             | 0  | 0 | 0 | 1 | 1              | 0              | 0              | 0              | 0              | 1              | 0              |   |  |
|               |   |   |   | 1             | 0 | 0 | 1 | 0 | 1             | 0  | 0 | 1 | 1 | 1              | 0              | 0              | 1              | 0              | 1              | 1              | 0 |  |
|               |   |   |   | 1             | 0 | 1 | 0 | 1 | 1             | 0  | 1 | 0 | 1 | 1              | 0              | 1              | 0              | 1              | 0              | 1              | 0 |  |
|               |   |   |   | 1             | 0 | 1 | 1 | 0 | 1             | 0  | 1 | 0 | 1 | 1              | 0              | 1              | 0              | 1              | 1              | 1              | 0 |  |
|               |   |   |   | 1             | 1 | 0 | 0 | 0 | 1             | 1  | 0 | 0 | 0 | 1              | 1              | 0              | 0              | 0              | 1              | 1              | 1 |  |

Table 1. Continuation

|  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|--|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|  | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 0 |
|  | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
|  | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |

To design circuits with numbers 1, 2, 3 patterns having 25 gates (5.5) were chosen, and for the circuit number 4, the pattern possessed 16 gates (4.4). Comparative results of design using traditional methods and using methods based on genetic algorithms NGA, and MGA are taken from [7]. Results obtained by those methods and by MLCEA method for circuits fulfilling the truth tables from Table 1 are presented in Table 2 (sign “~” in function description represents negation).

Table 2. Comparison results for designed circuits

| Method | Obtained function - circuit number 1                                                                                                                                | Number of gates |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| KM     | $F = Z \cdot (X \oplus Y) + Y \cdot (X \oplus Z)$                                                                                                                   | 5               |
| QM     | $F = \sim X \cdot Y \cdot Z + X \cdot (Y \oplus Z)$                                                                                                                 | 6               |
| NGA    | $F = (Z + Y) \cdot \sim (Y \oplus (X \oplus Z))$                                                                                                                    | 5               |
| MGA    | $F = (X + Y) \cdot Z \oplus (X \cdot Y)$                                                                                                                            | 4               |
| MLCEA  | $F = (X \cdot Z) \oplus (Y \cdot (X + Z))$                                                                                                                          | 4               |
| Method | Obtained function - circuit number 2                                                                                                                                | Number of gates |
| KM     | $F = ((\sim Z \cdot X) \oplus (\sim Y \cdot \sim W)) + ((\sim X \cdot Y) \cdot (Z \oplus \sim W))$                                                                  | 11              |
| NGA    | $F = \sim ((W \cdot Y \cdot \sim X) \oplus ((W + Y) \oplus Z \oplus (X + Y + Z)))$                                                                                  | 10              |
| MGA    | $F = \sim (((W \oplus Y) + (W \cdot X)) \oplus ((Z + X + Y) \oplus Z))$                                                                                             | 8               |
| MLCEA  | $F = ((X + Y) \cdot Z) \oplus (((Y \cdot X) + W) \oplus \sim X)$                                                                                                    | 7               |
| Method | Obtained function - circuit number 3                                                                                                                                | Number of gates |
| KM     | $F = ((A \oplus B) \oplus ((A \cdot D)(B + C))) + \sim ((A + C) + D)$                                                                                               | 9               |
| QM     | $F = \sim (\sim B \cdot (\sim A \cdot (D + C) + C \cdot D) + (A \cdot B \cdot \sim D))$                                                                             | 12              |
| NGA    | $F = ((B \oplus A) \oplus (A \cdot D)) + \sim (C + (D \oplus A))$                                                                                                   | 7               |
| MGA    | $F = ((A \oplus B) \oplus (A \cdot D)) + \sim (C + (A \oplus D))$                                                                                                   | 7               |
| MLCEA  | $F = ((\sim D \cdot A) \oplus B) + ((\sim D \oplus A) \cdot \sim C)$                                                                                                | 7               |
| Method | Obtained function - circuit number 4                                                                                                                                | Number of gates |
| KM     | $X_0 = A_0 \oplus B_0; X_1 = (A_1 \oplus B_1) \cdot \sim B_0 + ((A_1 \oplus B_1) \oplus A_0) \cdot B_0;$<br>$X_2 = A_1 \cdot B_1 + A_0 \cdot B_0 \cdot (A_1 + B_1)$ | 12              |
| NGA    | $X_0 = A_0 \oplus B_0; X_1 = A_0 \cdot B_0 \oplus (A_1 \oplus B_1);$<br>$X_2 = (A_0 \cdot B_0) \cdot (A_1 \oplus B_1) + (A_1 \cdot B_1)$                            | 7               |
| MGA    | $X_0 = A_0 \oplus B_0; X_1 = A_0 \cdot B_0 \oplus (A_1 \oplus B_1);$<br>$X_2 = (A_1 \cdot B_1) + A_0 \cdot B_0 \cdot (A_1 \oplus B_1)$                              | 7               |
| MLCEA  | $X_0 = A_0 \oplus B_0; X_1 = (A_1 \oplus B_1) \oplus (A_0 \cdot B_0);$<br>$X_2 = (A_1 \oplus B_1) \cdot (A_0 \cdot B_0) + (A_1 \cdot B_1)$                          | 7               |

## 4 Conclusions

Circuits designed by MLCEA method consist of lower number of gates, than circuits designed using Karnaugh Maps and Quine-McCluskey methods. Compared to NGA and MGA systems, the circuits designed using MLCEA were in 3 cases better and in 5 cases (on 8 possible) comparable. No one solution obtained using MLCEA method was worse than solutions obtained by other methods.

## References

1. M. Karnaugh, "A Map Method for Synthesis of Combinational Logic Circuits", *Transaction of the AIEE, Communications and Electronic*, 72(I):593-599, November, 1953
2. W. V. Quine, "A Way to Simplify Truth Function", *American Mathematical Monthly*, 62(9):627-631, 1955
3. E. J. McCluskey, "Minimization of Boolean Function", *Bell Systems Technical Journal*, 35 (5): 1417-1444, November 1956
4. J. Miller, T. Kalganova, N. Lipnitskaya, D. Job, "The Genetic Algorithm as a Discovery Engine: Strange Circuits and New Principles", In *Proceedings of the AISB Symposium on Creative Evolutionary Systems (CES'99)*, Edinburgh, UK, 1999
5. T. Kalganova and J. Miller, "Evolving more efficient digital circuits by allowing circuit layout and multi-objective fitness", *Proceedings of the First NASA/DoD Workshop on Evolvable Hardware*, pages 54-63, Los Alamitos, California, 1999
6. C. A. Coello, A. D. Christiansen, and A. H. Aguirre, "Automated Design of Combinational Logic Circuits using Genetic Algorithms", *Proceedings of the International Conference on Artificial Neural Nets and Genetic Algorithms*, pages 335-338, April, 1997
7. C. A. Coello, A. H. Aguirre, and B. P. Buckles, "Evolutionary Multiobjective Design of Combinational Logic Circuits", *Proceedings of the Second NASA/DoD Workshop on Evolvable Hardware*, pages 161-170, Los Alamitos, California, July 2000
8. J. D. Schaffer, "Multiple Objective Optimization with Vector Evaluated Genetic Algorithms", In *Genetic Algorithms and their Applications: Proceedings of the First International Conference on Genetic Algorithms*, pages 93-100, Lawrence Erlbaum, 1985
9. C. A. Coello, A. D. Christiansen, A. H. Aguirre, "Use of Evolutionary Techniques to Automate the Design of Combinational Circuits", *International Journal of Smart Engineering System Design*, 2000